

Système de Reconnaissance Faciale

Projet de Fin d'Études — Sonatel Academy (Orange Digital Center, Dakar)

Auteur : Ibrahima Gabar Diop · [GitHub](#) · [Repo](#)

Table des matières

1. [Présentation](#)
 2. [Architecture](#)
 3. [Dataset](#)
 4. [Résultats des approches](#)
 5. [Installation](#)
 6. [Configuration](#)
 7. [Utilisation](#)
 8. [API interne](#)
 9. [Éthique & sécurité](#)
 10. [Licence](#)
-

1. Présentation

Système de reconnaissance faciale en temps réel basé sur **InsightFace buffalo_I** — pipeline deux étapes combinant RetinaFace (détection) et ArcFace ResNet50 (reconnaissance par embeddings). Aucun entraînement requis : le système reconnaît toute personne dont une photo de référence est disponible.

Évolution des approches

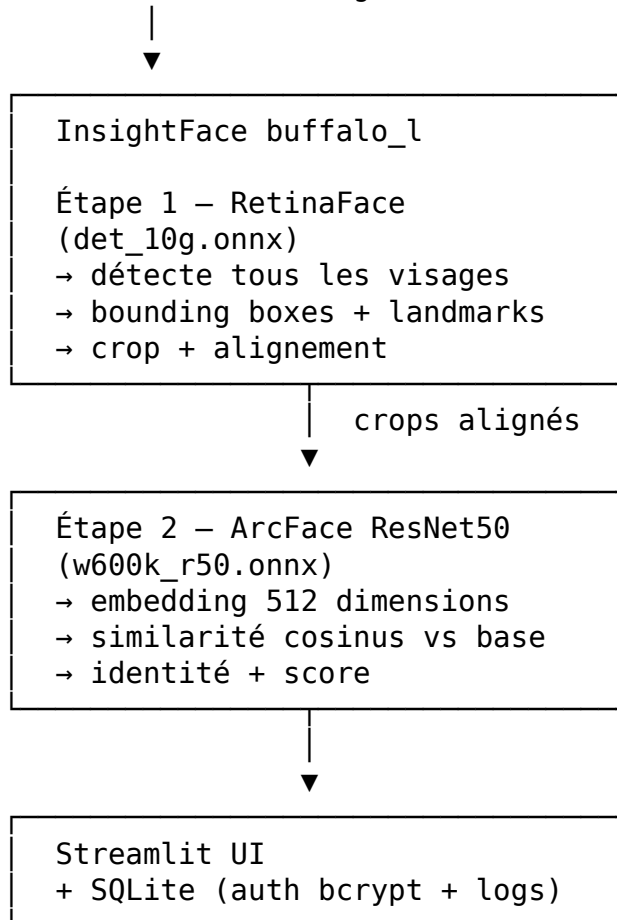
Approche	Précision	Limites
CNN from scratch	33 %	Pas de généralisation
ResNet-50 Transfer Learning	35 %	Idem
ResNet-50 Fine-tuning	40 %	Lent, limité aux classes entraînées
ResNet-50 Fine-tuning optimisé	53 %	2 h 30, toujours limité
YOLOv8 (détection + classification 1 passe)	~97 % sur LFW	Ne fonctionne pas sur flux réel
InsightFace buffalo_I (final)	État de l'art	Open-set, zéro réentraînement

Le passage à InsightFace a été décisif : les approches précédentes entraînaient un

classificateur fermé (N classes fixes). InsightFace utilise des **embeddings** — la reconnaissance fonctionne pour n'importe qui, il suffit d'une photo de référence.

2. Architecture

Entrée (webcam / image / vidéo)



Composants

Couche	Technologie	Rôle
Détection	RetinaFace (InsightFace)	Localisation + alignement des visages
Reconnaissance	ArcFace ResNet50 (InsightFace)	Embeddings 512-dim + similarité cosinus
Base de référence	Fichiers JPG dans data/faces/	1 photo minimum par identité
Cache embeddings	data/faces/embeddings.pk	Recalculé si data/faces/ change

Couche	Technologie	Rôle
Présentation	Streamlit + WebRTC	UI webcam live / image / vidéo
Persistance	SQLite + bcrypt	Utilisateurs, logs de reconnaissance

Pourquoi InsightFace buffalo_l

- **RetinaFace** : détecteur multi-échelle entraîné sur WiderFace (32k images), robuste aux occlusions, angles et faible éclairage
- **ArcFace** : perte angulaire qui maximise la séparabilité inter-classe — 99,77 % sur LFW benchmark
- **Open-set** : pas de réentraînement pour ajouter une nouvelle identité
- **Modèles ONNX** : inférence CPU/GPU sans dépendance PyTorch

3. Dataset

Source : [Labeled Faces in the Wild \(LFW\)](#)

Utilisé pour construire la base de référence (embeddings des 20 personnes les plus représentées).

LFW Top-20

#	Identité	Images
1	George W Bush	530
2	Colin Powell	236
3	Tony Blair	144
4	Donald Rumsfeld	121
5	Gerhard Schroeder	109
6	Ariel Sharon	77
7	Hugo Chavez	71
8	Junichiro Koizumi	60
9	Jean Chretien	55
10	John Ashcroft	53
11	Serena Williams	52
12	Jacques Chirac	52
13	Vladimir Putin	49
14	Lleyton Hewitt	41
15	Luiz Inacio Lula da Silva	41
16	Gloria Macapagal Arroyo	36
17	Andre Agassi	36
18	Laura Bush	35
19	Winona Ryder	31

#	Identité	Images
20	Jennifer Capriati	30

Total : ~1 900 images

Génération de la base de référence (Kaggle)

Le notebook `kaggle_kernel/train_yolo.ipynb` : 1. Charge les images LFW Top-20 2. Génère un embedding ArcFace moyen par personne (moyenne de 5 images) 3. Exporte `face_embeddings.npz` + une image de référence par personne 4. Ces fichiers se déposent dans `data/faces/` — reconnaissance immédiate au démarrage

4. Résultats des approches

Benchmark ArcFace (LFW officiel)

Modèle	LFW Accuracy
VGG-Face	98,95 %
FaceNet	99,63 %
ArcFace ResNet50	99,77 %
AdaFace R100	99,82 %

Performances système (CPU local)

Source	FPS estimé	Latence par frame
Image (upload)	—	~200-500 ms
Vidéo	~3-5 FPS	~200-300 ms
Webcam live (1/5 frames)	Flux 30 FPS	~200 ms par analyse

5. Installation

Prérequis

- Python 3.10+
- (Recommandé) GPU CUDA pour l'inférence temps réel

Étapes

```
git clone https://github.com/Gblack98/Gblack98-Systeme-de-Reconnaissance-Faciale.g
cd Gblack98-Systeme-de-Reconnaissance-Faciale

python -m venv .venv
source .venv/bin/activate          # Windows : .venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env
```

Au premier démarrage, InsightFace télécharge automatiquement les modèles buffalo_l (~300 MB).

```
streamlit run streamlit_app.py
```

Ajouter les identités LFW (optionnel)

Lancer kaggle_kernel/train_yolo.ipynb sur Kaggle, puis déposer les fichiers générés :

```
data/faces/George_W_Bush.jpg
data/faces/Colin_Powell.jpg
... (20 images de référence)
```

Le cache d'embeddings (embeddings.pkl) se reconstruit automatiquement.

Dépendances principales

```
insightface>=0.7.3
onnxruntime>=1.16.0
streamlit>=1.35
streamlit-webrtc>=0.47
opencv-python-headless>=4.9
bcrypt>=4.1
python-dotenv>=1.0
pandas>=2.0
```

6. Configuration

Fichier .env (copier depuis .env.example) :

Variable	Défaut	Description
DB_PATH	data/users.db	Base SQLite utilisateurs + logs
FACES_DB_PATH	data/faces	Dossier des images de référence

Variable	Défaut	Description
RECOGNITION_THRESHOLD	0.45	Seuil similarité cosinus ArcFace (0-1)
WEBCAM_PROCESS_EVERY_N	5	Analyser 1 frame sur N (webcam live)

Régler RECOGNITION_THRESHOLD : - 0.55+ → strict, peu de faux positifs - 0.45 → bon équilibre (défaut) - 0.35 → permissif, meilleur rappel

7. Utilisation

Interface Streamlit

Reconnaissance (📷) — image ou vidéo uploadée

Webcam (📷) — flux live via WebRTC, analyse toutes les N frames

Ajouter un visage (📷) — déposer une photo, embeddings recalculés immédiatement

Historique (📷) — 200 dernières reconnaissances de l'utilisateur connecté

Ajouter une identité

1. Aller dans 📷 Ajouter un visage
2. Saisir le nom (format Prénom_Nom)
3. Uploader une photo claire, face visible
4. Reconnaissance active immédiatement — pas de redémarrage requis

8. API interne

app/recognition.py

```
def detect_and_recognize(frame: np.ndarray) -> list[dict]:
    """
    Pipeline complet : RetinaFace détection + ArcFace reconnaissance.
    Retourne :
        [ {"bbox": (x, y, w, h), "identity": str,
          "confidence": float, "verified": bool} ]
    """

def draw_results(frame: np.ndarray, detections: list[dict]) -> np.ndarray:
    """Annote le frame avec bounding boxes et labels."""

def rebuild_face_db() -> None:
    """Invalide le cache et reconstruit la base d'embeddings."""
```

app/database.py

```
def init_db() -> None
def register_user(first_name, last_name, username, password) -> bool
def authenticate_user(username, password) -> dict | None
def log_recognition(user_id: int, identity: str, confidence: float) -> None
def get_recognition_logs(user_id: int) -> list[tuple]
```

9. Éthique & sécurité

Limitations

- **Dataset biaisé** — LFW surreprésente les hommes politiques occidentaux (2002–2004)
- **Conditions dégradées** — occultation, profil, très faible éclairage réduisent les performances
- **Open-set** — toute personne sans photo de référence retourne “Inconnu”

Usage responsable

Système développé à des **fins académiques**. Tout déploiement en production requiert : - Respect du **RGPD** (ou législation locale sur la biométrie) - **Consentement explicite** des personnes identifiées - Interdiction d’usage discriminatoire

Sécurité du code

- Mots de passe hashés **bcrypt**
 - Aucun credential dans le code — variables d’environnement via `.env`
 - `data/users.db` et `data/faces/` exclus du versioning
-

10. Licence

MIT License — Copyright (c) 2024 Ibrahima Gabar Diop

Développé par Ibrahima Gabar Diop — Sonatel Academy, Orange Digital Center, Dakar — 2024